

CS-2303

System Programming Concepts

Hugh C. Lauer
Adjunct Professor

(Slides include materials from *The C Programming Language*, 2nd edition, by Kernighan and Ritchie and from *C: How to Program*, 5th and 6th editions, by Deitel and Deitel)

Introduction

- Survey and TA/SA Introductions
- Pause to Look Backwards and Forwards
- Course Objectives
- Course Operation/Expectations
- Course Plan and Syllabus
- Systems Concepts
- ‘Old’ Development Environment
 - C and C++
- Higher Level Language History

Look Backwards and Forwards

- Computers
- WPI CS Curriculum
- Instructor
- Students
 - Expected Background
 - Going Forward
 - Your Future

CS2303 Course Objectives

- To expose students to lower level systems interface 'grunge' only clearly visible via C.
- To learn to program in C++ by learning to program in C first.
- To further develop the ability to design programs with emphasis on the abstract view of data structures.
- To get experience with the low-level implementation of data structures in C.

CS2303 Course Objectives

- To learn the advantages of programming in an object-oriented language such as C++.
- To experience programming in the

Large

CS2303 Course Objectives

Pointers!!

Course Operation/Expectations

- The course web site:–

<http://web.cs.wpi.edu/~cs2303/c10/>

- Contains:–

- Syllabus
- General overview
- Rules and expectations
- All lecture notes
- All programming and lab assignments
- ...

You are responsible for everything
on the course web site

Course Operation/Expectations (continued)

- One lab per week
 - Required
- Approx 6 Programming Assignments
- 2 Exams
 - Mid-term:– week of Feb 8 (approx)
 - Final:– March 5

Grading

- Exams:– ~40%
- Programming Assignments:– ~40%
- Labs and Subject Evaluation:– ~20%

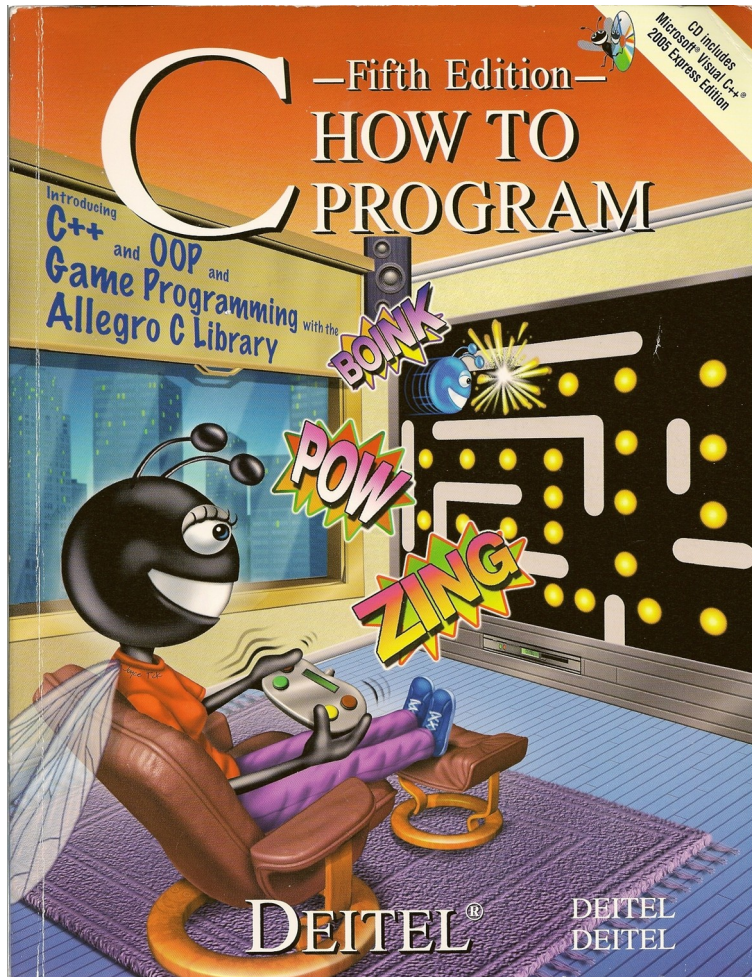
Satisfactory grades on Programming Assignments
are required to pass this course

It is in your interest that the Professor and TAs
know who you are!

Course Plan and Syllabus

- Approx 50% C, approx 50% C++
- To cover the details of C briskly.
 - Assume an understanding of iteration and conditional constructs — similar to Java
- To introduce data structures in C via
 - Arrays and pointers (*very different from Java*).
 - Structs and unions
- {Note - reading of the textbook will require jumping around during the C portion of the course.}

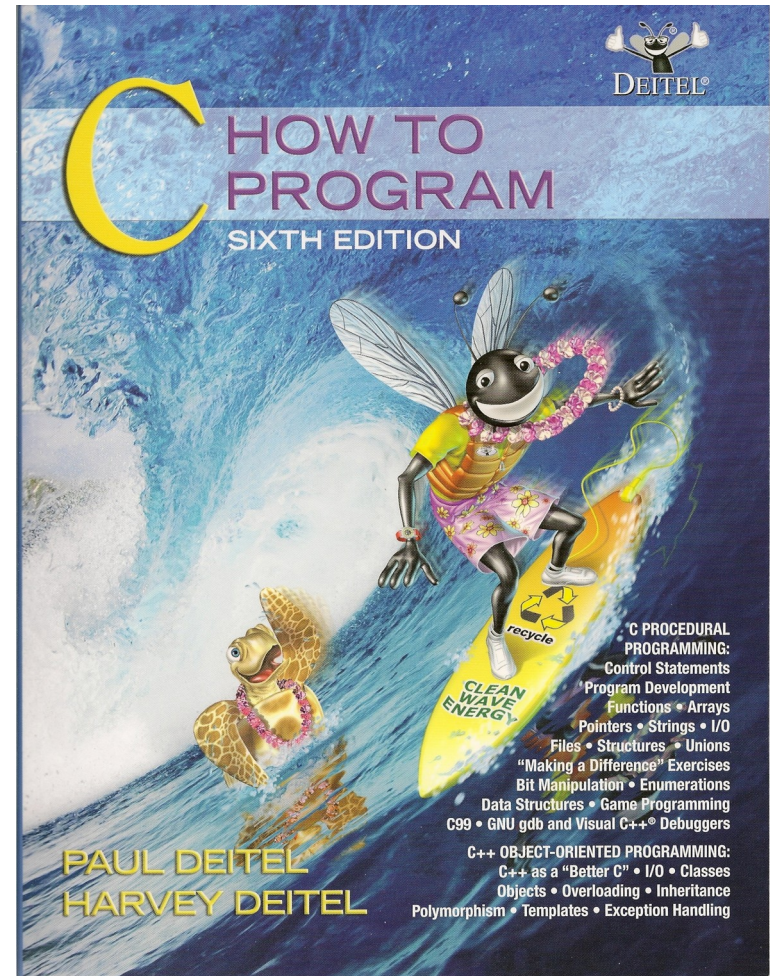
Required Textbook



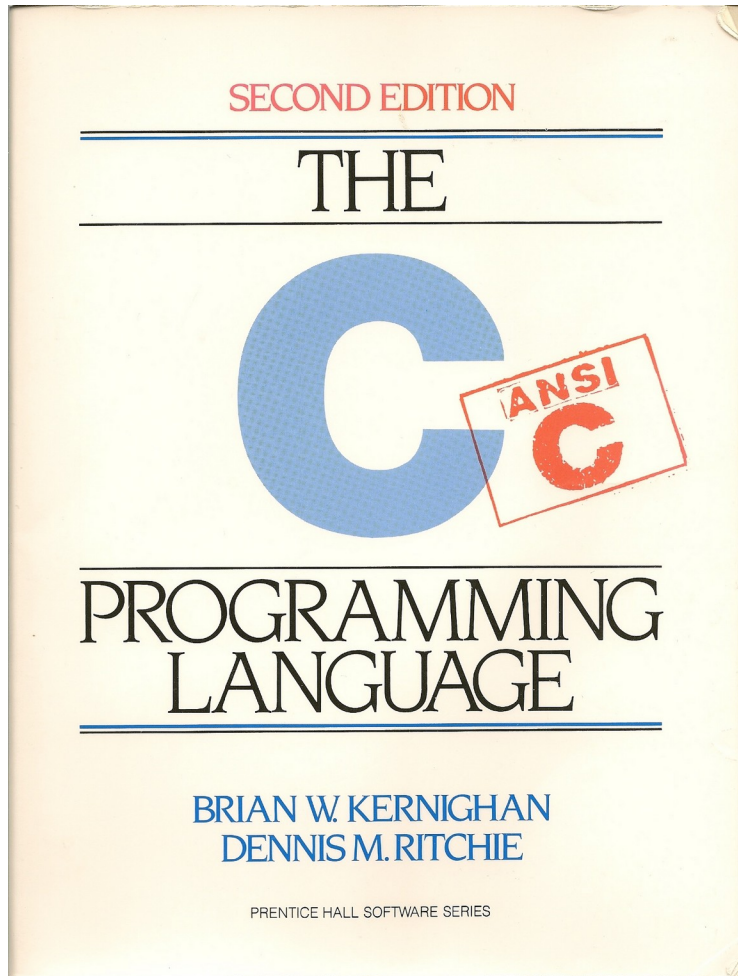
- Deitel & Deitel, 5th ed.
- C & C++
- Went out of print at end of 2009!
- Bookstore has enough copies for CS-2303!

Alternative Textbook

- Deitel & Deitel, 6th ed.
- C & C++
- Similar to 5th edition.
- C++ chapters have different numbers.
- *Note: all references to pages, figures, and examples will be relative to 5th edition!*



Highly Recommended for CS Majors



- Over 20 years old!
- Still relevant
 - *Will refer to it often!*
- Cheap
 - Used on Amazon
 - Download PDF
- Keep a copy on your desk ...
- ... for the rest of your professional life!

Questions?

Your First Programming Assignment

- *Due:* next Thursday, Jan 21, 11:59 PM!
- Create a 12-month calendar for an arbitrary year!
- Full details on course web site:—
 - *Programming Assignments* → *Scroll to bottom* → *PA1*
- *Reading:*— D&D, Chapters 2, 3, 4
 - This part of the *C* language is a lot like *Java*
 - Except **printf()**, **scanf()**
 - See D&D Chapter 9 or K&R Chapter 7 for details

Why C?

- Because we have to!
- Many situations where it is *only* language or system available
 - Small, embedded systems, instrumentation, etc.
- Many “low-level” situations that don’t have support for “high-level” languages
 - Operating systems, real-time systems, drivers

Why not C?

- C is very low-level
 - Data structures must be programmed “by hand”
 - Operations must be done out in “long hand”
 - No support for “object oriented” design
 - Marginal support for higher-level thought processes
 - *Much, much harder to use than higher level languages/systems*
- Better alternatives available for almost all applications
 - *Java, Python, Ruby, etc.* – many CS situations
 - *Matlab, SimuLink* – physical modeling
 - *LabView* – instrumentation and control
 - *Excel* – accounting and statistics
 - *SQL* – billing and transactions
 - ...

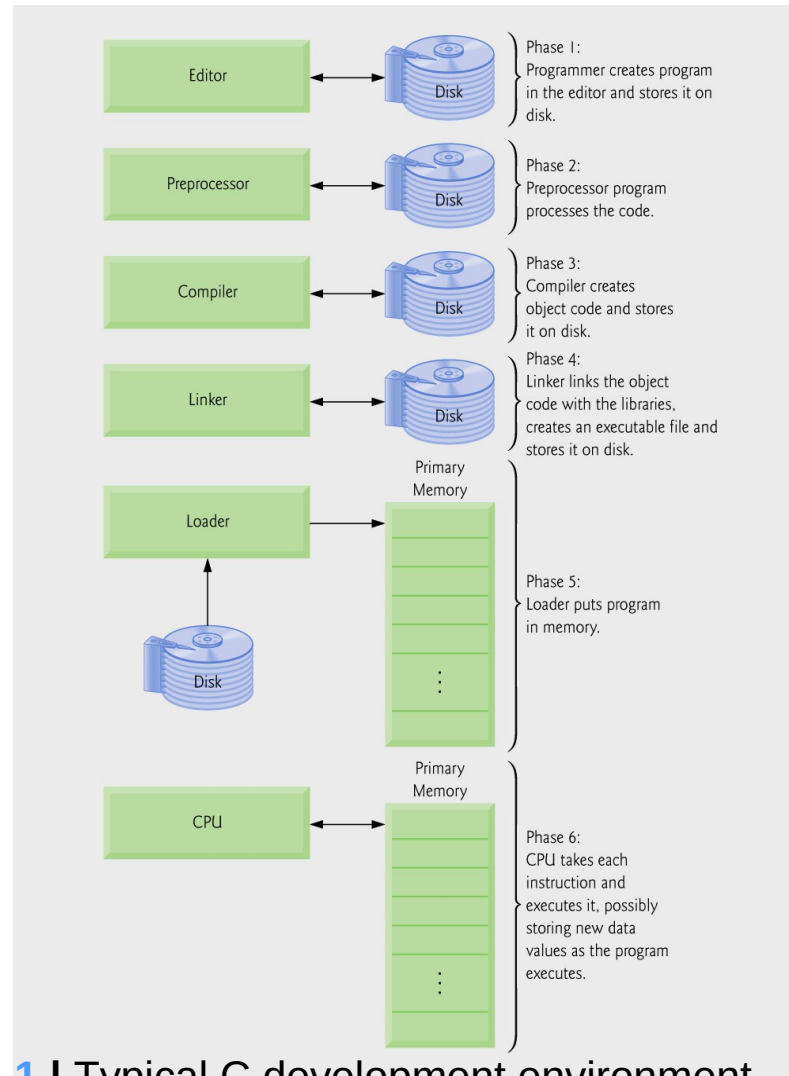
What about C++?

- Object-oriented thinking
 - Data abstractions, classes, objects, interfaces
 - Operator overloading
 - Inheritance
 - Lots of other good stuff
 - ...
- Backward compatible with C
 - To some extent
 - Allows programmer to get close to hardware when needed
 - Allows programmer to get close to data representation when needed
 - *Not* platform independent (like *Java*)
 - Still need to be conscious of memory management
 - ...

C Program Development Environment

Standard Steps

1. *Edit*
2. *Preprocess*
3. *Compile*
4. *Link*
5. *Load*
6. *Execute*



Deitel & Deitel

A Short History

- *In the beginning ...*

- Machine language
- Assembly language
 - One line per machine instruction

Really primitive!

Too difficult for big projects

- So “high level” languages were invented

- Non-recursive:– Fortran, Cobol
- Recursive:– Algol, Lisp, Snobol, PL/1, etc.

Too advanced!

Too *much* infrastructure for operating systems, control systems, many kinds of projects

A Short History (continued)

- 1960s:– “system programming” languages invented
 - More direct control over hardware
 - Not so primitive as Assembly Language
- E.g., a veritable alphabet soup
 - BLISS, BCPL, PL/360, ...

A Short History (continued)

- Late 1960s:– MIT's *MULTICS* project doesn't live up to expectation
 - Bell Labs wants to pull out
 - Ken Thompson starts on *Unix* as alternative
- Thompson creates own system language
 - Based on BCPL, untyped, called *B*
 - First version of Unix written in *B*
- Unix takes hold inside Bell Labs
 - Needs to be re-written
 - Dennis Ritchie *et al* create *C* as typed successor to *B*

A Short History (continued)

- *C* wins the race of “system programming” languages
 - Language of choice for many operating systems
 - Many applications
- Object-oriented programming begins to emerge
 - Stroustrup starts on *C with Classes*
 - Backward compatible with *C*
 - Later renamed *C++*

A Short History (continued)

- By 1990, C++ is language of choice for many kinds of applications
 - *Good*:– object-oriented, classes, inheritance, etc.
 - *Bad*:– all of the ugly characteristics of C still persist
 - Complex object model, memory management, etc.
- Jim Gosling at Sun Microsystems starts a complete rewrite of C++
 - Simpler object model
 - Cleaner
 - Portable, can be embedded in web pages, etc.
 - Eventually called *Java*

Questions?